

❑ Uniwatch

A developer-first, unified infrastructure monitoring and self-healing platform — connect any server in minutes, not hours.

❑ What is Uniwatch?

Uniwatch turns raw infrastructure into a live observability layer — no agents to manually install, no YAML hell, no \$500/mo SaaS bills. Drop in a server IP and SSH key, and within seconds:

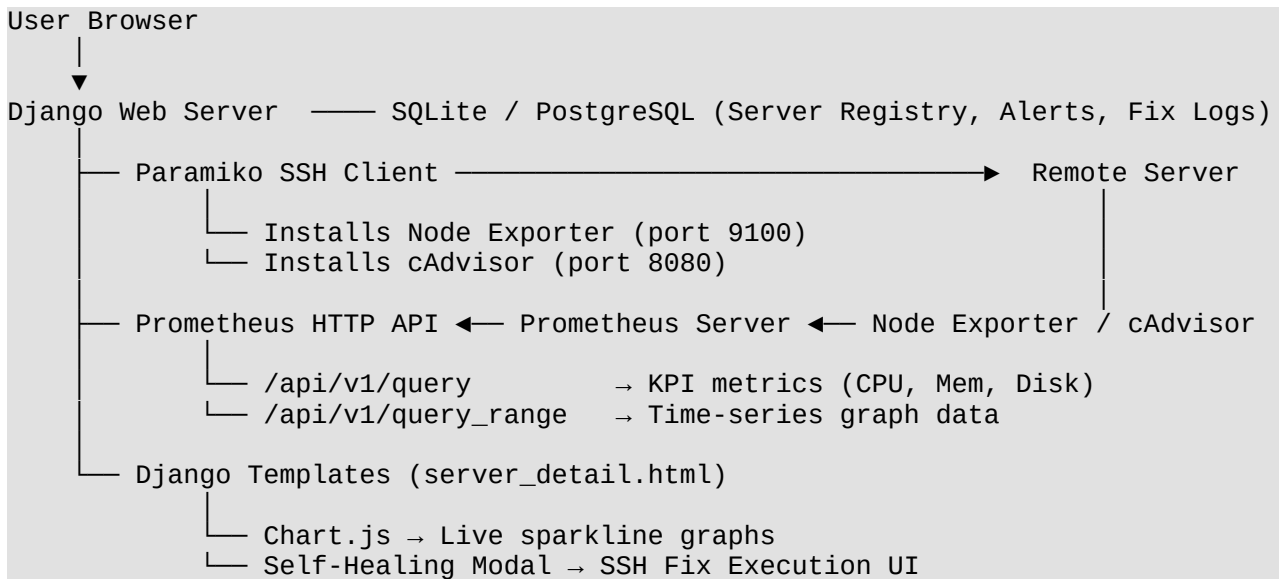
- Monitoring tools are **auto-deployed** remotely
- Metrics are **streaming live** to your dashboard
- Alerts **fire automatically** when thresholds are crossed
- A **self-healing engine** can diagnose and remediate issues via SSH — with full audit logs

Built for DevOps engineers, startup teams, and indie hackers who need production-grade visibility without enterprise complexity.

❑ Features

Feature	Description
Agentless Onboarding	Add any server using just an IP address and SSH key. Nothing to pre-install.
⚙️ Auto-Deployment	Automatically installs Prometheus Node Exporter and cAdvisor over SSH via Paramiko.
📊 Real-Time Metrics	Live CPU, Memory, Disk, Network I/O, Load Average, TCP connections, and IOPS.
Container Monitoring	Full cAdvisor integration for per-container resource tracking.
Multi-Condition Alerts	Alert rules evaluate CPU (>60%), Memory (>85%), Disk (>90%), and Network spikes.
Self-Healing Engine	Suggests and executes whitelisted fix commands (cache clear, log rotate, zombie kill) via SSH.
⚡ Auto-Fix Mode	Toggle per-server automatic remediation for critical alerts.
🔍 Execution Audit Log	Every fix attempt is recorded with command output, exit codes, and timestamps.
Smart Insights	Contextual recommendations based on actual metric patterns.

🏗️ Architecture Overview



Components

Component	Role
Django 4.2	Web framework, SSH orchestration, API endpoints, template rendering
Paramiko	SSH key parsing and remote command execution
Prometheus	Time-series metric collection via file-based service discovery
Node Exporter	Exposes host-level metrics (CPU, disk, memory, network)
cAdvisor	Exposes per-container metrics via Docker
Chart.js	Client-side real-time graph rendering
SQLite / PostgreSQL	Persists server registry, alert records, and fix execution logs

⚙️ How It Works

Step 1 — Add a Server

User provides a server name, IP address, SSH username, and private key. The key is used **once and never stored**.

Step 2 — Auto-Deploy Monitoring Tools

Django connects over SSH via Paramiko and runs a pre-built command sequence:

- Creates a `node_exporter` system user
- Downloads and installs the Prometheus Node Exporter binary
- Creates and enables a `systemd` service
- Optionally installs cAdvisor as a Docker container for container monitoring

Step 3 — Prometheus Scrapes Metrics

The server is registered in Prometheus's file-based service discovery (`targets/uniwatch_targets.json`). Prometheus begins scraping metrics from port 9100 (and 8080 for containers) within seconds.

Step 4 — Dashboard Visualizes Data

The Django backend queries Prometheus's HTTP API (`/api/v1/query` and `/api/v1/query_range`) to hydrate the dashboard with live KPIs and 30-minute time-series graphs across 4 categories:

- System Performance (CPU, Load Averages)
- Memory Utilization (Usage %, Breakdown)
- Storage & Disk I/O (Usage %, IOPS, Throughput)
- Network Activity (RX/TX MB/s, TCP Connections)

Step 5 — Alerts Fire

Every page load evaluates metric thresholds:

Metric	Condition	Severity
CPU Usage	> 60% (1-min avg)	Critical
Memory Usage	> 85%	Critical
Disk Usage	> 90%	⚠ Warning
Network RX	> 100 MB/s	⚠ Warning

Step 6 — Self-Healing Engine Acts

Each alert displays a "**Fix Now**" button. Clicking it:

1. Fetches a preview of whitelisted safe commands from `fix_actions.py`
2. User reviews commands and provides SSH key
3. Commands execute sequentially over SSH with 30s timeout and 1 auto-retry
4. Full `stdout`, `stderr`, exit codes, and retry counts are displayed

5. Execution is logged to the `FixExecution` database table with status and timestamp

Auto-Fix Mode can be toggled per-server to bypass manual approval for critical alerts.

☐ Screenshots

(Add screenshots of the running application here)

View	Description
<code>dashboard.png</code>	Main server dashboard with KPIs and accordion-grouped graph categories
<code>alerts-panel.png</code>	Active alert cards with Fix Now buttons and severity indicators
<code>fix-modal.png</code>	Self-healing command preview modal with SSH key input
<code>fix-logs.png</code>	Post-execution command output, exit codes, and result status
<code>landing.png</code>	Marketing landing page with product description and CTAs

✂ Tech Stack

Backend

- **Python 3.9+**
- **Django 4.2** — Web framework and HTTP API
- **Paramiko** — SSH client for remote command execution
- **Requests** — Prometheus API HTTP client

Frontend

- **Vanilla JS + Django Templates** — Server-side rendered UI
- **Chart.js** — Real-time sparkline and time-series graphs
- **CSS Custom Properties** — Deep dark design system with neon accents

Monitoring

- **Prometheus** — Metric collection and storage
- **Node Exporter v1.7.0** — Host-level metrics
- **cAdvisor** — Container metrics via Docker

Storage

- **SQLite** (default) / **PostgreSQL** (production)

Infrastructure

- **Paramiko** — SSH connection and remote execution transport
-

✂ Setup Instructions

Prerequisites

- Python 3.9+
 - Prometheus running locally (see below)
 - A remote Linux server accessible via SSH
-

1. Clone the Repository

```
git clone https://github.com/your-username/univatch.git
cd univatch/Uniwatch1
```

2. Backend Setup

```
# Create and activate a virtual environment
python3 -m venv venv
source venv/bin/activate          # On Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Apply database migrations
python3 manage.py migrate

# (Optional) Create a superuser for Django Admin
python3 manage.py createsuperuser

# Start the development server
python3 manage.py runserver 8001
```

The application will be available at: **http://127.0.0.1:8001/**

3. Prometheus Setup

```
# Install Prometheus (macOS)
brew install prometheus

# Or download manually from https://prometheus.io/download/
```

Create a `prometheus.yml` config that uses file-based service discovery:

```
global:
```

```
scrape_interval: 15s

scrape_configs:
- job_name: 'node_exporters'
  file_sd_configs:
  - files:
    -
'/path/to/univatch/Uniwatch1/prometheus/targets/uniwatch_targets.json'
  refresh_interval: 10s

# Start Prometheus
prometheus --config.file=prometheus.yml
```

Prometheus UI will be at: **http://localhost:9090/**

4. Add Your First Server

1. Open **http://127.0.0.1:8001/**
2. Click "**Get Started Free**"
3. Fill in: Server Name, IP Address, SSH Username
4. Paste your SSH private key (RSA, Ed25519, or ECDSA supported)
5. Click "**Deploy Monitoring**"

Monitoring agents deploy in ~30 seconds. The dashboard goes live automatically.

☐ Security Considerations

SSH Key Handling

- Private keys are held **in memory only** during setup/remediation and immediately garbage collected
- Keys are **never written to disk or stored in the database**
- Paramiko's `AutoAddPolicy` is used with full awareness — future versions will implement known-hosts pinning

Self-Healing Command Safety

- All executable commands are defined in `monitor/fix_actions.py` — **no dynamic or user-generated commands can ever be executed**
- Every command is validated against a `DANGEROUS_PATTERNS` blacklist (blocks `rm -rf /`, `mkfs`, `shutdown`, `fork bombs`, etc.) as a defense-in-depth layer
- Commands run with a hard `30-second timeout` per execution
- Failed commands retry a maximum of **1 time** before marking as failed

Audit Logging

- Every fix execution attempt is persisted to the `FixExecution` database model
 - Fields captured: server, alert reference, commands run (JSON), stdout, exit code, `triggered_by` (`manual` / `auto`), timestamp
 - Accessible via Django Admin at `/admin/`
-

☐ Future Improvements

- ☐ **AI-Powered Insights** — Use LLM to analyze metric patterns and suggest architectural improvements
 - ☐ **Multi-Tenant Architecture** — Per-user server isolation with authentication (JWT/OAuth)
 - ☐ **Kubernetes Monitoring** — kube-state-metrics and kubelet cadvisor integration
 - ☐ **Slack / PagerDuty Alerts** — Webhook-based notification routing for critical events
 - ☐ **Metric Retention Policies** — Configurable time-range retention for Prometheus TSDB
 - ☐ **RBAC** — Role-based access control for fix approvals in team environments
 - ☐ **WebSocket Streaming** — Push-based real-time graph updates instead of polling
 - ☐ **Ansible Playbook Integration** — Full Ansible support for complex provisioning workflows
-

☐ Contributing

Contributions are welcome! Here's how to get started:

1. **Fork** the repository
2. **Create a feature branch:** `git checkout -b feature/your-feature-name`
3. **Commit your changes:** `git commit -m "feat: add your feature"`
4. **Push to the branch:** `git push origin feature/your-feature-name`
5. **Open a Pull Request** with a clear description of your changes

Guidelines

- Follow the existing code style (PEP 8 for Python)
- Do **not** add arbitrary command execution to `fix_actions.py` — all new fix commands must be safe, reversible, and explicitly justified
- Write clear commit messages using [Conventional Commits](#) format
- Update the README if you add a significant new feature

□ License

This project is licensed under the **MIT License**. See [LICENSE](#) for details.

Built with ♥ by Suryansh Saxena

"Detect → Decide → Act"